Linear Regression

Christos Dimitrakakis

October 14, 2026

Outline

The Linear Model
- Simple linear regression
- History
- Multiple linear regression

Optimisation algorithms
- Gradient Descent
- Least-Squares

Regression libraries in Python
- sklearn
- statsmodels

Problems
- Interpretation Problem parameters
- Exercises

The Linear Model
- Simple linear regression
- History
- Multiple linear regression

Optimisation algorithms
- Gradient Descent
- Least-Squares

Regression libraries in Python
- sklearn
- statsmodels

Problems
- Interpretation Problem parameters
- Exercises

Simple linear regression

Input and output

- Data pairs (x^t, y^t), $t = 1, \ldots, T$.
- Input $x^t \in \mathbb{R}$
- Output $y^t \in \mathbb{R}$.

Modelling the conditional expectation $\mathbb{E}[y^t|x^t]$

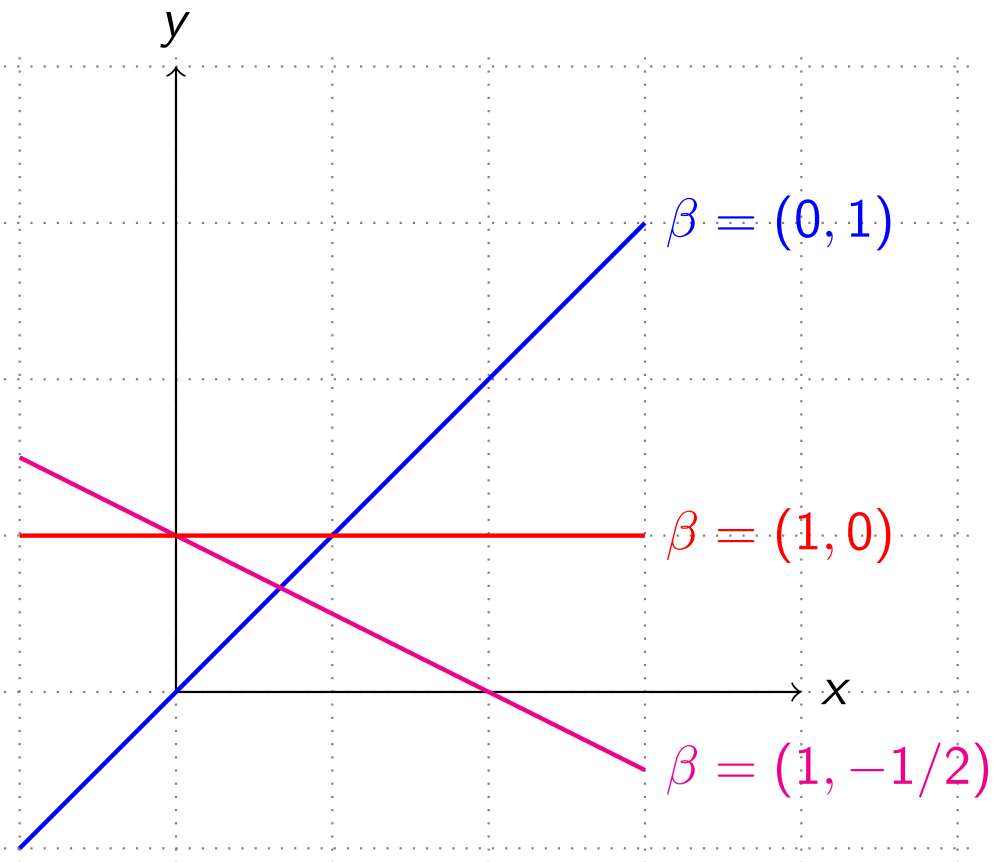
- Parameters $\beta_0, \beta_1 \in \mathbb{R}$
- Function $\pi_\beta : \mathbb{R} \to \mathbb{R}$, defined as

$$\pi_\beta(x^t) = \beta_0 + \beta_1 x_t$$

Probabilistic predictions: Modelling the conditional probability $\mathbb{P}[y^t|x^t]$

- $y^t = \mathbb{E}[y^t|x^t] + \epsilon_t$, with $\epsilon_t \in \mathbb{R}$ being zero-mean noise.
- Simplest model: variance $\sigma = \mathbb{V}(\epsilon) = \mathbb{E}[\epsilon_t^2]$

Linear models



$$\pi_\beta(x) = \beta_0 + \beta_1 x = [\beta_0, \beta_1] \begin{bmatrix} 1 \\ x \end{bmatrix}$$

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y

Learning as inference

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.

Learning as inference

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.
- We can minimise the average loss over the data

Learning as inference

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.
- We can minimise the average loss over the data
- Ideally, we want to minimise the expected loss, with respect to the unknown distribution P.

Learning as inference

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.
- We can minimise the average loss over the data
- Ideally, we want to minimise the expected loss, with respect to the unknown distribution P.

Learning as inference

- The parameters β define a probabilistic model $P_\beta(y|x)$ for every value of y.

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.
- We can minimise the average loss over the data
- Ideally, we want to minimise the expected loss, with respect to the unknown distribution P.

Learning as inference

- The parameters β define a probabilistic model $P_\beta(y|x)$ for every value of y.
- We want to find the parameters giving the highest probability on the observed data.

Two views of the problem

Learning as optimisation

- Each value $\pi_\beta(x)$ is a prediction about the value of y
- We suffer a loss $\ell(y, \pi_\beta(x))$ for every example (x, y) that we see.
- We can minimise the average loss over the data
- Ideally, we want to minimise the expected loss, with respect to the unknown distribution P.

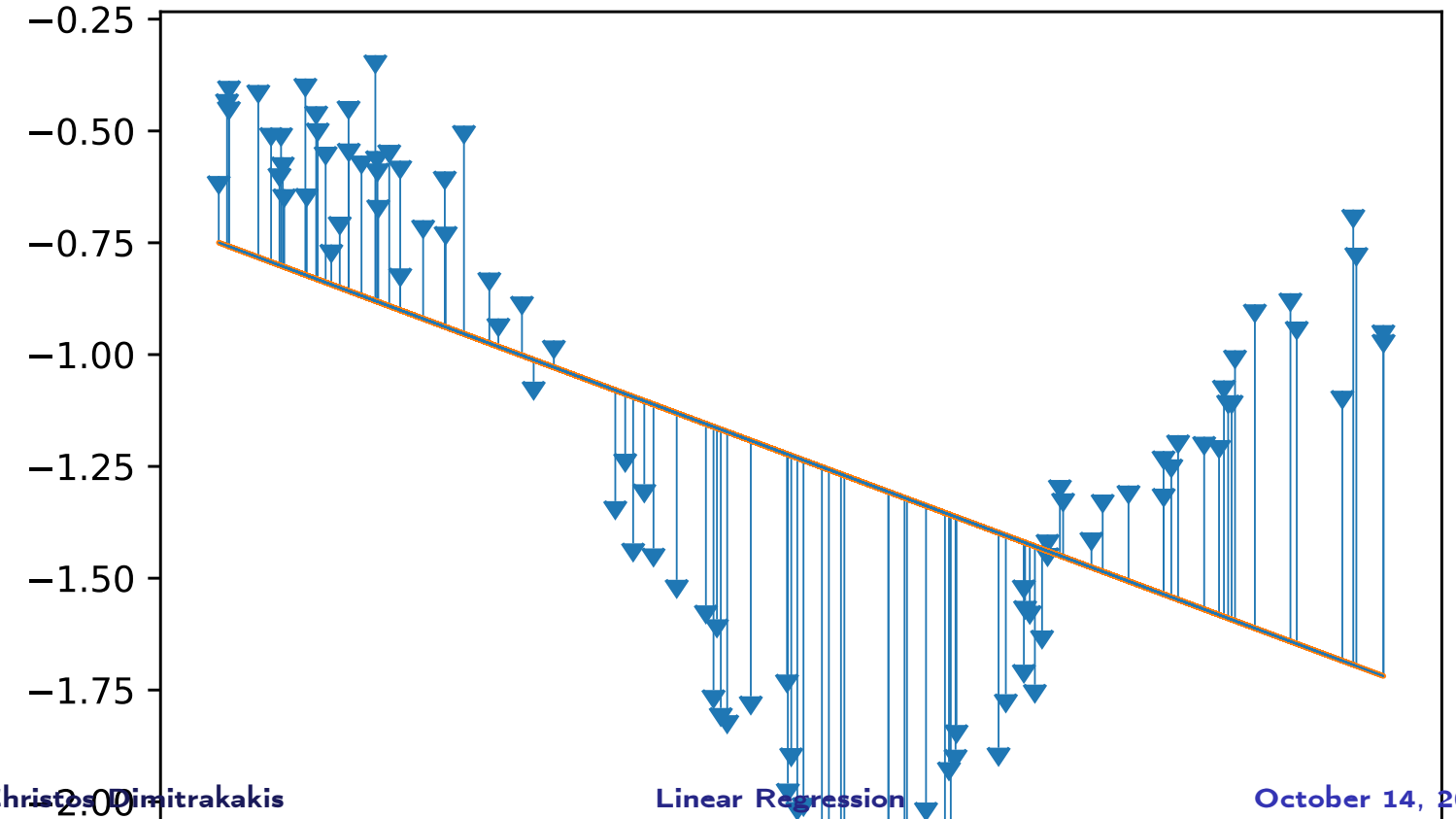
Learning as inference

- The parameters β define a probabilistic model $P_\beta(y|x)$ for every value of y.
- We want to find the parameters giving the highest probability on the observed data.
- Ideally, we want to find the true conditional distribution $P(y|x)$.

Learning as Optimisation

Find the parameters β minimising squared error

$$\min_{\beta} \frac{1}{T} \sum_{t=1}^{T} \Big[\underbrace{y^t - \pi_\beta(x^t)}_{\text{residual}} \Big]^2$$



Origins

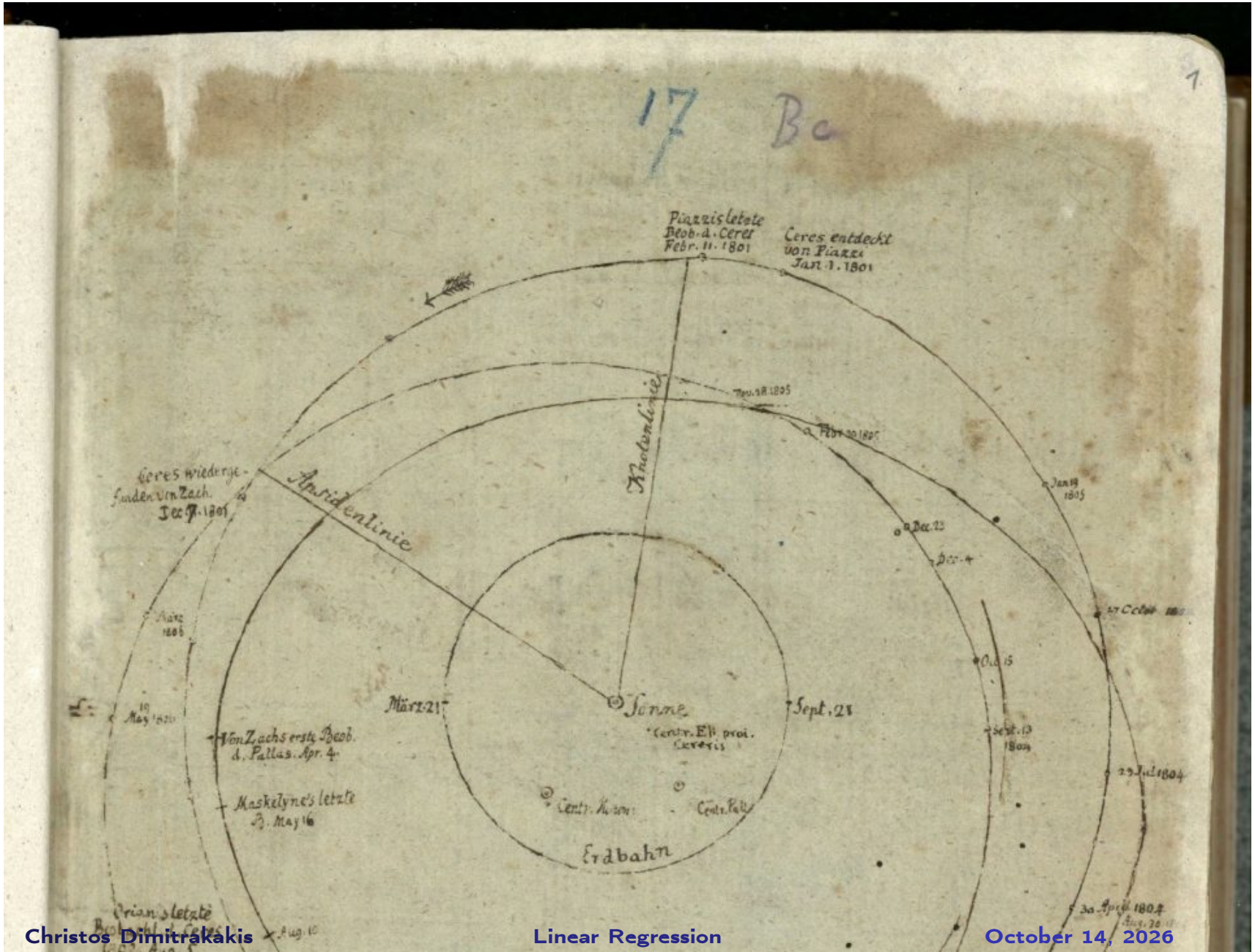


Figure: Gauss: originator

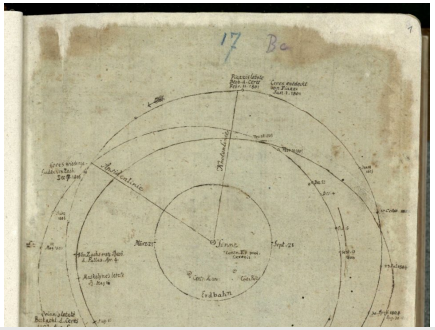


Figure: Legendre: first publication

The orbit of Ceres



The orbit of Ceres



On Gauss and Legendre. Gauss applied the method of least squares to calculate the orbit of Ceres, an asteroid detected in the solar system. At the time, astronomers were keen to establish what the orbit was, but they did not even know where to point their telescopes. He was the only one who predicted the orbit sufficiently well to tell them where to look. He apparently had already developed the method when he was aged 18, but he only published it much later. He also, of course, famously defined the normal (Gaussian) distribution, which is closely linked to the least-squares method.

The full story of Gauss, Laplace, Legendre and Ceres.

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{ \exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right) \right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference

Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference

Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}
\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum \ln p_\beta(y^t|x^t) \\
&= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\
&= \arg\max_\beta \sum -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum [y^t - \pi_\beta(x^t)]^2
\end{aligned}$$

Maximum likelihood inference
Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference
Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \ldots, y^t|x^1, \ldots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\ &= \arg\max_\beta \sum_t \ln p_\beta(y^t|x^t) \\ &= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\ &= \arg\max_\beta \sum_t -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum_t |y^t - \pi_\beta(x^t)|^2\end{aligned}$$

Gaussian noise model. In regression, we can never exactly predict y^t. In terms of loss minimisation, we are simply trying to minimise some measure of error in our predictions. However, an alternative approach is to assume that y^t is inherently unpredictable, but that it follows a specific distribution, with a mean centered around the prediction $\pi_\beta(x^t)$. Mathematically, we express this by saying that y^t is the result of adding some noise ϵ_t to the output of the mean function $\pi(x^t)$,

$$y^t = \pi_\beta(x^t) + \epsilon_t.$$

Since we have to assume something about the noise, a common assumption is that it is normally distributed. We write this as

$$\epsilon_t \sim \text{Normal}(0, \sigma),$$

where $\text{Normal}(0, \sigma)$ means the normal distribution with mean 0 and variance σ^2. We can also write this in terms of a conditional density function.

$$p_\beta(y^t|x^t)i \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2),$$

where the proportionality symbol hides the constant $1/\sqrt{2\pi\sigma^2}$.

While the Gaussian noise model is not the only possibility, it has some interesting properties.

Maximum likelihood inference
Gaussian noise model.

$$y^t = \pi(x^t) + \epsilon_t, \qquad \epsilon_t \sim \text{Normal}(0, \sigma)$$

With conditional density:

$$p_\beta(y^t|x^t) \propto \exp(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2)$$

Maximum likelihood inference
Idea: For data D, find parameters maximising $P_\beta(D)$

$$\begin{aligned}\arg\max_\beta P_\beta(D) &= \arg\max_\beta p_\beta(y^1, \dots, y^t|x^1, \dots, x^t) = \arg\max_\beta \ln \prod_t p_\beta(y^t|x^t) \\ &= \arg\max_\beta \sum_t \ln p_\beta(y^t|x^t) \\ &= \arg\max_\beta \sum_t \ln \left\{\exp\left(-[y^t - \pi_\beta(x^t)]^2/2\sigma^2\right)\right\} \\ &= \arg\max_\beta \sum_t -[y^t - \pi_\beta(x^t)]^2/2\sigma^2 = \arg\min_\beta \sum_t |y^t - \pi_\beta(x^t)|^2\end{aligned}$$

Maximum likelihood. Now that we have a probabilistic model, we can assign probabilities to the data we have. This is the central idea of the maximum likelihood principle:

- For data D, find parameters β maximising $P_\beta(D)$.

Proof explanation.

Let us apply this principle to our case. Here, first of all, we only care about predicting y from x. For that reason, we define

$$P_\beta(D) p_\beta(y^1, \dots, y^t|x^1, \dots, x^t)$$

We also assume that y^t only depends on x^t if we know the mean and variance, so that $p_\beta(y^1, \dots, y^t|x^1, \dots, x^t) = \prod_t p_\beta(y^t|x^t)$.
The maximum point of some function $f(\beta)$ is the same of some monotonically increasing transformation g of it, i.e. if $f(\beta^*) \geq f(\beta)$, then $g[f(\beta^*)] \geq g[f(\beta)]$.
Hence, we can maximise $\ln \prod_t p_\beta(y^t|x^t)$ instead. And since $\ln(a+b) = \ln(a)\ln(b)$.
Finally, note that if $\exp(a) = b$ then $a = \ln(b)$.

What this means

So, the β maximising the likelihood with a Gaussian noise model, is the same β that maximises the squared error. So the two different approaches lead to the same result.

Coding break

src/Regression/RegressionDemonstration.ipynb

- Show implementation
- Fit and residuals
- Multiple draws from the distribution
- Fit on non-linear data?

Multiple linear regression

Input and output

- Data pairs (x^t, y^t), $t = 1, \ldots, T$.
- Input $x^t \in \mathbb{R}^n$
- Output $y^t \in \mathbb{R}^m$.

Multiple linear regression

Input and output

- Data pairs (x^t, y^t), $t = 1, \ldots, T$.
- Input $x^t \in \mathbb{R}^n$
- Output $y^t \in \mathbb{R}^m$.

Point predictions: Modelling the conditional expectation $\mathbb{E}[y^t|x^t]$

- Parameters $\boldsymbol{B} \in \mathbb{R}^{n \times m}$
- Function $\pi_{\boldsymbol{B}} : \mathbb{R}^n \to \mathbb{R}^m$, defined as

$$\pi_\beta(x^t) = \boldsymbol{B}^\top x_t = \sum_{i=1}^{n} \beta_i x_{t,i}$$

Probabilistic predictions: Modelling the conditional probability $\mathbb{P}[y^t|x^t]$

- $y^t = \mathbb{E}[y^t|x^t] + \epsilon_t$, with $\epsilon_t \in \mathbb{R}^m$ being zero-mean noise.
- Noise covariance matrix $\Sigma = \mathbb{V}(\epsilon) = \mathbb{E}[\epsilon_t \mid \epsilon_t \top]$

The Linear Model
- Simple linear regression
- History
- Multiple linear regression

Optimisation algorithms
- Gradient Descent
- Least-Squares

Regression libraries in Python
- sklearn
- statsmodels

Problems
- Interpretation Problem parameters
- Exercises

Gradient descent algorithm

Minimising a function

$$\min_{\beta} f(\beta) \leq f(\beta') \forall \beta', \qquad \beta^* = \arg\min_{\beta} f(\beta) \Rightarrow f(\beta^*) = \min_{\beta} f(\beta)$$

Gradient descent algorithm

Minimising a function

$$\min_{\beta} f(\beta) \leq f(\beta') \forall \beta', \qquad \beta^* = \arg\min_{\beta} f(\beta) \Rightarrow f(\beta^*) = \min_{\beta} f(\beta)$$

Gradient descent for minimisation

- Input β_0
- For $n = 0, \ldots, N$:
- $\beta_{n+1} = \beta_n - \eta_n \nabla_\beta f(\beta_n)$

Gradient descent algorithm

Minimising a function

$$\min_{\beta} f(\beta) \leq f(\beta') \forall \beta', \qquad \beta^* = \arg\min_{\beta} f(\beta) \Rightarrow f(\beta^*) = \min_{\beta} f(\beta)$$

Gradient descent for minimisation

- Input β_0
- For $n = 0, \ldots, N$:
- $\beta_{n+1} = \beta_n - \eta_n \nabla_\beta f(\beta_n)$

Step-size η_n

- η_n fixed: for online learning
- $\eta_n = c/[c + n]$ for asymptotic convergence
- $\eta_n = \arg\min_\eta f(\beta_n + \eta \nabla_\beta)$: Line search.

Gradient descent for squared error

The cost function

$L(\beta, D) = \sum_{t=1}^{T}(y^t - \pi_\beta(x^t))^2 = \sum_{t=1}^{T} \epsilon_t^2$, with $\epsilon_t \triangleq y^t - \pi_\beta(x^t)$.

Gradient descent for squared error

The cost function

$L(\beta, D) = \sum_{t=1}^{T}(y^t - \pi_\beta(x^t))^2 = \sum_{t=1}^{T} \epsilon_t^2$, with $\epsilon_t \triangleq y^t - \pi_\beta(x^t)$.

Cost gradient

Using the chain rule of differentiation, $\nabla_\beta f(\epsilon) = \nabla_\epsilon f(\epsilon) \nabla_\beta \epsilon$.

$$\nabla_\beta L(\beta, D) = \nabla_\beta \sum_{t=1}^{T} \epsilon_t^2 = \sum_{t=1}^{T} \nabla_\beta \epsilon_t^2 = \sum_{t=1}^{T} \nabla_{\epsilon_t} \epsilon_t^2 \nabla_\beta \epsilon$$

$$= \sum_{t=1}^{T} 2\epsilon_t \nabla_\beta [y^t - \pi_\beta(x^t)] = \sum_{t=1}^{T} 2[y^t - \pi_\beta(x^t)][-\nabla_\beta \pi_\beta(x^t)]$$

Gradient descent for squared error

The cost function

$L(\beta, D) = \sum_{t=1}^{T}(y^t - \pi_\beta(x^t))^2 = \sum_{t=1}^{T} \epsilon_t^2$, with $\epsilon_t \triangleq y^t - \pi_\beta(x^t)$.

Cost gradient

Using the chain rule of differentiation, $\nabla_\beta f(\epsilon) = \nabla_\epsilon f(\epsilon) \nabla_\beta \epsilon$.

$$\nabla_\beta L(\beta, D) = \nabla_\beta \sum_{t=1}^{T} \epsilon_t^2 = \sum_{t=1}^{T} \nabla_\beta \epsilon_t^2 = \sum_{t=1}^{T} \nabla_{\epsilon_t} \epsilon_t^2 \nabla_\beta \epsilon$$

$$= \sum_{t=1}^{T} 2\epsilon_t \nabla_\beta [y^t - \pi_\beta(x^t)] = \sum_{t=1}^{T} 2[y^t - \pi_\beta(x^t)][-\nabla_\beta \pi_\beta(x^t)]$$

Parameter gradient for linear regression

Remember $\nabla_\beta f = (\partial/\partial_1 f, \ldots, \partial/\partial_n f)$

$$\frac{\partial}{\partial \beta_j} \pi_\beta(x^t) = \frac{\partial}{\partial \beta_j} \sum_{i=1}^{n} \beta_i x_{t,i} = \sum_{i=1}^{n} \frac{\partial}{\partial \beta_j} \beta_i x_{t,i} = x_{t,j}.$$

Stochastic gradient descent algorithm

When f is an expectation

$$f(\beta) = \int_X dP(x) g(x, \beta).$$

Replacing the expectation with a sample:

$$\begin{aligned} \nabla f(\beta) &= \int_X dP(x) \nabla g(x, \beta) \\ &\approx \frac{1}{K} \sum_{k=1}^{K} \nabla g(x^{(k)}, \beta), \qquad x^{(k)} \sim P. \end{aligned}$$

Stochastic gradient descent algorithm

When f is an expectation

$$f(\beta) = \int_X dP(x) g(x, \beta).$$

Replacing the expectation with a sample:

$$\begin{aligned} \nabla f(\beta) &= \int_X dP(x) \nabla g(x, \beta) \\ &\approx \frac{1}{K} \sum_{k=1}^{K} \nabla g(x^{(k)}, \beta), \qquad x^{(k)} \sim P. \end{aligned}$$

Note For the general case, we got to do this.

Analytical Least-Squares Solution

We need to solve the following equations for A:

$$\begin{aligned} y^1 &= (\boldsymbol{x}^1)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^t &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^T &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \end{aligned}$$

Analytical Least-Squares Solution

We need to solve the following equations for A:

$$\begin{aligned} y^1 &= (\boldsymbol{x}^1)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^t &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^T &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \end{aligned}$$

We can rewrite it in matrix form:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_t \\ \vdots \\ y_T \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_t^\top \\ \vdots \\ \boldsymbol{x}_T^\top \end{pmatrix} \boldsymbol{\beta}$$

Analytical Least-Squares Solution

We need to solve the following equations for A:

$$\begin{aligned} y^1 &= (\boldsymbol{x}^1)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^t &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^T &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \end{aligned}$$

We can rewrite it in matrix form:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_t \\ \vdots \\ y_T \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_t^\top \\ \vdots \\ \boldsymbol{x}_T^\top \end{pmatrix} \boldsymbol{\beta}$$

Resulting in

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta}.$$

Analytical Least-Squares Solution

We need to solve the following equations for A:

$$\begin{aligned} y^1 &= (\boldsymbol{x}^1)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^t &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \\ \cdots &\quad \cdots \\ y^T &= (\boldsymbol{x}^t)^\top \boldsymbol{\beta} \end{aligned}$$

We can rewrite it in matrix form:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_t \\ \vdots \\ y_T \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_t^\top \\ \vdots \\ \boldsymbol{x}_T^\top \end{pmatrix} \boldsymbol{\beta}$$

Resulting in

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta}.$$

How can we get $\boldsymbol{\beta}$?

Finding the β

We now have a linear equation,

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta}.$$

We want to solve for β. If $\boldsymbol{X}$ had an inverse $\boldsymbol{X}^{-1}$, we could obtain

$$\boldsymbol{X}^{-1}\boldsymbol{y} = \boldsymbol{X}^{-1}\boldsymbol{X}\boldsymbol{\beta} = I\boldsymbol{\beta} = \boldsymbol{\beta}.$$

But $\boldsymbol{X}^{-1}$ does not exist.

Finding the β

We now have a linear equation,

$$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta}.$$

We want to solve for β. If $\boldsymbol{X}$ had an inverse $\boldsymbol{X}^{-1}$, we could obtain

$$\boldsymbol{X}^{-1}\boldsymbol{y} = \boldsymbol{X}^{-1}\boldsymbol{X}\boldsymbol{\beta} = I\boldsymbol{\beta} = \boldsymbol{\beta}.$$

But $\boldsymbol{X}^{-1}$ does not exist.

Least-squares solution

The left-pseudo inverse $\tilde{\boldsymbol{X}}^{-1} \triangleq (\boldsymbol{X}^\top \boldsymbol{X})^{-1}\boldsymbol{X}^\top$ can be used to obtain

$$\boldsymbol{\beta} = \tilde{\boldsymbol{X}}^{-1}\boldsymbol{y},$$

This follows as:

$$\begin{aligned}
\boldsymbol{y} &= \boldsymbol{X}\boldsymbol{\beta} \\
\tilde{\boldsymbol{X}}^{-1}\boldsymbol{y} &= \tilde{\boldsymbol{X}}^{-1}X\boldsymbol{\beta} \\
\tilde{\boldsymbol{X}}^{-1}\boldsymbol{y} &= \underbrace{(\boldsymbol{X}^\top \boldsymbol{X})^{-1}}_{\boldsymbol{A^{-1}}}\underbrace{\boldsymbol{X}^\top \boldsymbol{X}}_{\boldsymbol{A}}\boldsymbol{\beta}.
\end{aligned}$$

Some matrix algebra reminders

The identity matrix $\boldsymbol{I} \in \mathbb{R}^{n \times n}$

- For this matrix, $I_{i,i} = 1$ and $I_{i,j} = 0$ when $j \neq i$.
- $\boldsymbol{I}\boldsymbol{x} = \boldsymbol{x}$ and $\boldsymbol{I}\boldsymbol{A} = \boldsymbol{A}$.

Some matrix algebra reminders

The identity matrix $\boldsymbol{I} \in \mathbb{R}^{n \times n}$

- For this matrix, $I_{i,i} = 1$ and $I_{i,j} = 0$ when $j \neq i$.
- $\boldsymbol{I}\boldsymbol{x} = \boldsymbol{x}$ and $\boldsymbol{I}\boldsymbol{A} = \boldsymbol{A}$.

The inverse of a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$

$\boldsymbol{A}^{-1}$ is called the inverse of $\boldsymbol{A}$ if

- $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$.
- or equivalently $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$.

Some matrix algebra reminders

The identity matrix $\boldsymbol{I} \in \mathbb{R}^{n \times n}$

- For this matrix, $I_{i,i} = 1$ and $I_{i,j} = 0$ when $j \neq i$.
- $\boldsymbol{I}\boldsymbol{x} = \boldsymbol{x}$ and $\boldsymbol{I}\boldsymbol{A} = \boldsymbol{A}$.

The inverse of a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$

$\boldsymbol{A}^{-1}$ is called the inverse of $\boldsymbol{A}$ if

- $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$.
- or equivalently $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$.

The pseudo-inverse of a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times m}$

- $\tilde{\boldsymbol{A}}^{-1}$ is called the left pseudoinverse of $\boldsymbol{A}$ if $\tilde{\boldsymbol{A}}^{-1}\boldsymbol{A} = \boldsymbol{I}$.

$$\tilde{\boldsymbol{A}}^{-1} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1}\boldsymbol{A}^\top, \qquad n > m$$

- $\tilde{\boldsymbol{A}}^{-1}$ is called the right pseudoinverse of $\boldsymbol{A}$ if $\boldsymbol{A}\tilde{\boldsymbol{A}}^{-1} = \boldsymbol{I}$.

$$\tilde{\boldsymbol{A}}^{-1} = \boldsymbol{A}^\top (\boldsymbol{A}\boldsymbol{A}^\top)^{-1}, \qquad m > n$$

The Linear Model
- Simple linear regression
- History
- Multiple linear regression

Optimisation algorithms
- Gradient Descent
- Least-Squares

Regression libraries in Python
- sklearn
- statsmodels

Problems
- Interpretation Problem parameters
- Exercises

sklearn

Fitting a model to data

```
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X, Y)
```

Getting predictions

We can get predictions for all inputs as an array

```
Z = model.predict(X)
```

Statsmodels

Fitting a model to data X, Y

```
import statsmodels.api as sm
Xa = sm.add_constant(X) # adds a constant factor to the data
model = sm.OLS(Y, Xa)
results = model.fit()
```

Getting predictions

The prediction is not just a point!

```
z = results.get_prediction(Xa[t])
z.predicted_mean # This is E[y|x]
```

The Linear Model
- Simple linear regression
- History
- Multiple linear regression

Optimisation algorithms
- Gradient Descent
- Least-Squares

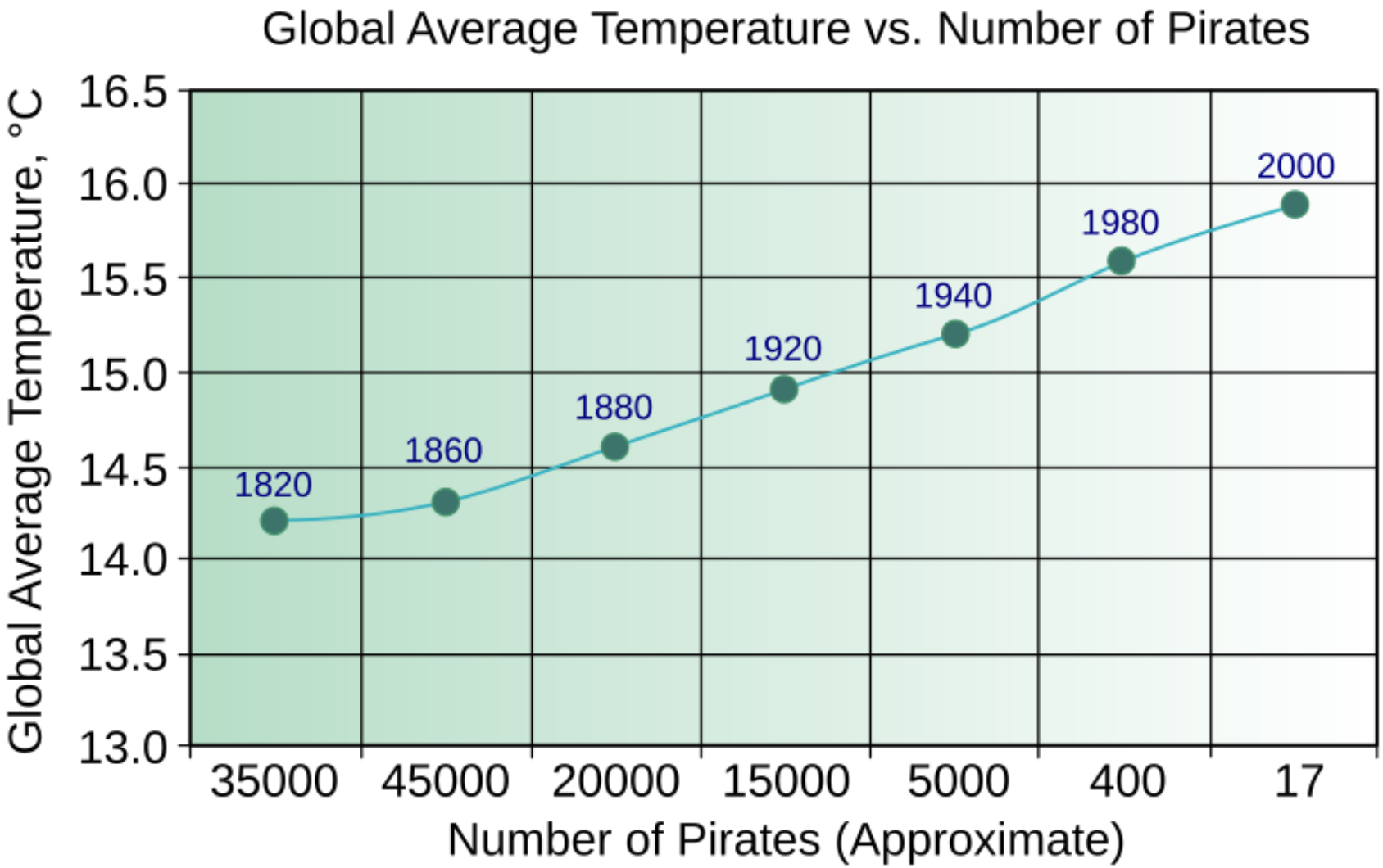
Regression libraries in Python
- sklearn
- statsmodels

Problems
- Interpretation Problem parameters
- Exercises

Pitfalls

- β_i tells us how much y is correlated with $x_{t,i}$
- However, multiple correlations might be evident.
- Some features may be irrelevant
- The relationship may not be linear
- Correlation is not causation

Correlation is not causation



Linear regression exercises

- Exercises 8, 13 from ISLP
- A variant of Ex. 13 but with Y generated independently of X.